

Informe Preliminar

Desafío I - Sweet Crush | Informática II

1. Introducción

Este proyecto consiste en desarrollar el motor lógico para un juego, llamado Sweet Crush. El objetivo del proyecto no será solo hacer que el juego funcione, sino aprender a gestionar los recursos de la computadora al nivel más bajo posible.

Por esta razón: no podemos usar herramientas automáticas que nos solucionen la vida (como la librería STL, clases u objetos), sino que debemos construir todo desde cero usando lógica pura. El núcleo del desafío es empaquetar la información del tablero en un flujo continuo de bits dentro de la memoria dinámica, obligándonos a dominar el uso de punteros y la reserva de memoria.

2. Análisis del Problema

El problema principal es que no podemos guardar las fichas en variables normales (como un arreglo de int). Hay que meter todas las fichas seguidas, una detrás de otra, usando solo los bits necesarios para ahorrar espacio.

Requisitos del Sistema

- **Crear el tablero:** El usuario escribe cuántas filas y columnas quiere. El programa calcula el espacio exacto en memoria y lo reserva al momento.
- **Tamaño de las fichas:** Cada ficha usa exactamente 3 bits. Hay 6 fichas diferentes y 2 estados especiales.
- **Memoria junta:** Las fichas van pegadas en los bytes. No se dejan espacios vacíos para alinearlas a un byte completo.
- **Manejo de memoria:** El tablero se guarda en la memoria dinámica usando punteros comunes y corrientes.
- **Cambiar tamaño:** El juego debe dejar meter o sacar filas y columnas en cualquier lado. Para cambiar el tamaño real de la memoria, se creará un bloque nuevo con `new [ ]`, se copiarán los datos y se borrará el viejo con `delete[ ]`. Esto solo se hace si el tablero queda con menos del 65% de uso.
- **Lógica del juego:** El programa tiene que hacer dos cosas. Primero, dejar que el jugador elija y borre una ficha. Segundo, revisar automáticamente si se forman líneas de 3 fichas iguales para borrarlas. Si algo se borra, las fichas de arriba deben "caer" por gravedad.

- **Bits sobrantes:** Como casi nunca va a dar un número exacto de bytes, los bits que sobren se van a dejar al final de la memoria. Así el tablero empieza limpio desde el primer bit del primer byte.

3. Diseño de la Solución

El código va a ser puramente procedimental (funciones sueltas) y lo vamos a repartir en varios archivos .h y .cpp.

Variables y Parámetros

No se van a usar variables globales para nada. El tablero, las filas, las columnas y el tamaño en bytes se van a pasar como argumentos en cada función. Si una función necesita cambiar el tamaño de la memoria o mover el tablero de lugar, recibirá un puntero a puntero (por ejemplo: unsigned char** tablero) para poder modificar la dirección desde adentro.

Mover filas y columnas

Mover datos a mitad del tablero es lo más difícil del proyecto. Si metemos o sacamos una fila en el medio, no podemos copiar bytes enteros porque las fichas ocupan 3 bits y quedan cortadas entre dos bytes. Hay que hacer un bucle que mueva los datos bit por bit desde el punto del cambio hacia adelante, usando máscaras para no romper los datos vecinos.

Códigos de las Fichas

- **000 al 101:** Los 6 tipos de fichas para jugar.
- **110:** Casilla vacía (sirve para ver dónde tienen que caer las fichas de arriba).
- **111:** Espacio reservado

Cascadas

1. **Buscar y marcar:** El programa recorre el tablero buscando líneas de 3 o más fichas iguales, en horizontal y vertical. Si encuentra una combinación, cambia el código de esas fichas a 110 (Celda vacía).
2. **Aplicar Gravedad:** El programa revisa el tablero. Si encuentra un espacio vacío, hace caer la ficha que esté justo arriba y deja el espacio vacío arriba. Esto se repite hasta que no queden huecos debajo de ninguna ficha.
3. **Rellenar el techo:** Las celdas que quedaron vacías en la fila de más arriba (fila 0) se rellenan con fichas nuevas generadas al azar

4. **¿Hubo cambios?:** Si en el paso 1 se borró algo, significa que el tablero cambió. Por lo tanto, el bucle vuelve a empezar desde el paso 1 para ver si las fichas que cayeron formaron una nueva combinación automática. Si ya no se forman más líneas de 3, el combo termina.

Fórmulas para buscar una ficha

Si queremos buscar la ficha en la fila f y columna c , hacemos estos cálculos:

- Posición de la ficha: $i = f * \text{columnas} + c$
- Bit donde empieza: $\text{bit_inicio} = i * 3$
- Byte donde está: $\text{post_byte} = \text{bit_inicio} / 8$
- Lugar dentro de ese byte = $\text{bit_inicio} \% 8$

Organización de los Módulos

- **Modulo memoria:** Se encarga de hacer el `new[ ]` y `delete[ ]` para agrandar o achicar el tablero según cuánta memoria libre quede.
- **Modulo bit:** Tiene las funciones `extraer_ficha` y `escribir_ficha`. Usa operaciones de bits como `&`, `|`, `~`, `<<` y `>>`.
- **Modulo lógica:** Busca las líneas de 3 fichas, hace que caigan por gravedad y rellena los huecos de arriba con fichas nuevas al azar.
- **Modulo estadísticas:** Lleva la cuenta de los puntos, cuántas combinaciones van y cuántas cascadas se hicieron. Modifica variables que le pasamos por referencia.
- **Modulo interfaz:** Dibuja el juego en la consola. Va a mostrar el tablero como caracteres normales para jugar, y abajo mostrará los bytes reales en binario para comprobar que todo está bien.